Global Backend Feature Flags (A.K.A. Main Flipt)

Responsável: @Alex Rios (Deactivated) / @bruno.szczuk - Platform Acceleration HUB

Este documento descreve as boas práticas e referências relacionadas ao uso inicial do **Flipt** como ferramenta de Feature Flags (FF) dentro da Stone. O objetivo é garantir que todos os times sigam um padrão de implementação seguro, confiável e escalável, respeitando as restrições iniciais de uso e promovendo a melhoria contínua.

Introdução

O **Flipt** é uma ferramenta de feature flags de código aberto que permite habilitar ou desabilitar recursos em tempo de execução, sem a necessidade de realizar novos deploys.

> Dentro da Stone, optamos por iniciar o uso dessa ferramenta de forma controlada, utilizando apenas flags do tipo **booleana** (on/off) e apenas para fluxos de **Backend**, para garantir segurança e clareza durante o período inicial de adoção em produção.

Objetivos

1. **Estabelecer padrões de uso** de Feature Flags na Stone, garantindo que todos os times sigam boas práticas de desenvolvimento e operação.
2. **Minimizar riscos** relacionados à introdução de novas features em produção, mantendo visibilidade e controle granular.
3. **Orientar sobre limitações iniciais** e maneiras de contorná-las em casos excepcionais.
4. **Referenciar** guias e tutoriais de utilização, assim como oturos links importantes.

Escopo

- Este documento se aplica a todos os times que utilizam o Flipt como ferramenta de Feature Flags na Stone.
- Abrange o processo de **criação**, **gerenciamento** e **remoção** de flags booleanas no Flipt.
- **Não** abrange o uso de **Variant Attachments** (ainda).

Diretrizes

Uso inicial limitado a Flags booleanas

Essa diretriz tem a intenção de reduzir complexidade, riscos e facilitar o aprendizado sobre a ferramenta.

Boas práticas:

1. **Nomenclatura objetiva**: nomeie as flags de forma objetiva e descritiva (ex.: `enable_new_checkout_flow`, `disable_old_logging_system`).
   a. Por questões de padronização vamos seguir com:
      i. as chaves (keys) das flags em ***snake_case.*** (camelCase não é reprovavel em um PR, mas já consideramos *deprecated*)
      ii. os nomes de formato livre, mas recomenda-se uma descrição com espaços. ex: `Enables the new checkout flow`
2. **Escopo restrito**: crie flags específicas para cada funcionalidade ou teste, evitando "flags genéricas" ou que causem confusão sobre o que está realmente habilitado ou desabilitado.
3. **Documente a razão de existência**: sempre registre qual é o propósito da flag, quem é o responsável pela criação e qual o prazo de remoção ou reavaliação.

Evitar uso de Variant Attachments

A funcionalidade de variants é mais complexa, exige maior cuidado em deploy e monitoramento e, portanto, será evitada durante o período de adoção inicial, reduzindo riscos de erros em produção.

> **Variant Attachments**
>
> Na primeira onda de uso não utilizamos Variant Attachments, mas caso a necessidade seja urgente, nesses casos, recomenda-se a **instância própria de Flipt** ou **adiconar feature files na sua própria aplicação** para isolar e gerenciar esse tipo de carga de trabalho avançada.
>
> Dessa forma, é possível isolar e controlar melhor os impactos dessas configurações.

Ciclo de vida das Flags

Ao criar uma nova flag, certifique-se de que todos os envolvidos (time de engenharia e stakeholders) estejam cientes das motivações e do funcionamento.

Siga o procedimento de *gradual rollout* (ex.: habilitar progressivamente para grupos de usuários ou serviços específicos) quando aplicável.

Acompanhe métricas e logs para garantir que a habilitação ou desabilitação de uma flag não afete negativamente a estabilidade ou a experiência do usuário.

Após a consolidação do recurso controlado pela flag, planeje sua remoção para evitar acumular “flags obsoletas” que aumentam a complexidade de manutenção sistema.

Governança e Responsabiliaddes

Platform Acceleration HUB é responsável por definir e manter padrões, fornecer suporte e garantir que a instância de Flipt esteja operando adequadamente.

Times de engenharia são responsáveis por criar, gerenciar e documentar as flags, além de monitorar os impactos em seus respectivos serviços e produtos.

Stakeholders/Times de Produto devem alinhar prazos e objetivos de cada flag, garantindo que haja visibilidade do status da feature controlada.

Processo de Evolução

Este documento deve ser revisado a cada trimestre ou sempre que houver mudanças significativas na forma como a Stone utiliza o Flipt.

Os times são encorajados a fornecer feedback e sugestões de melhoria para este guia de boas práticas, visando aprimorar a adoção de Feature Flags na empresa.

Próximos passos

Step	Status	Accountable
Guia rápido sobre Trunk-based Development	DONE	@Alex Rios (Deactivated)
Definir modelo de gestão do repositório	DONE	@Alex Rios (Deactivated)

Referências extras:

- Trunk Based Development - Speedrun
- Repositório: https://github.com/stone-payments/main-flipt Conectar a conta do Github
- Repositório de exemplo: https://github.com/stone-payments/flipt-examples Conectar a conta do Github

- Documentação do Flipt: https://docs.flipt.io/introduction

Apêndice A - GitOps

Para complementar as boas práticas de uso do Flipt na Stone, é importante destacar como o **GitOps** é incorporado ao fluxo de trabalho de feature flags. GitOps é um conjunto de práticas que coloca toda a configuração de aplicações e infraestrutura dentro de repositórios Git, permitindo que o repositório se torne a **fonte única de verdade** (Single Source of Truth) para qualquer mudança.

Quando integramos o gerenciamento de feature flags a esse modelo, cada alteração realizada nas flags é versionada, auditada e passa pelos mesmos processos de *code review* já estabelecidos na empresa. Assim, ganhamos:

1. **Confiabilidade**: qualquer alteração de configuração de flags passa pelo fluxo de revisão e é rastreável no Git, reduzindo o risco de configurações incorretas ou quebras em produção.
2. **Segurança**: o controle de acesso (via GitHub) garante que somente pessoas autorizadas possam aprovar e fazer o *merge* mudanças, tornando possível seguir políticas de segurança e governança corporativas.
3. **Auditabilidade**: todo histórico de criação, edição e remoção de flags fica registrado em commits e pull requests, simplificando investigações futuras e garantindo rastreabilidade.
4. **Performance e Escalabilidade com OCI**: o Flipt oferece suporte a diversos *backends* de armazenamento (local, Git, object storage e OCI). O uso de registro em padrão OCI (como Docker Registry) traz **ganhos expressivos de performance**, pois esses registries são altamente otimizados para armazenamento e distribuição de artefatos. Dessa forma, grandes repositórios podem ser evitados, e o fluxo de sincronização pode ser mais rápido e menos suscetível a falhas.

Como funciona o GitOps com Flipt

1. **Definição declarativa das flags**: em vez de configurar as flags diretamente na interface do Flipt, criamos arquivos de configuração (YAML) que descrevem cada flag, seja ela habilitada ou não, com suas regras de rollout, *segments*, etc.
2. **Commit e Pull Requests**: ao realizar qualquer mudança nessas configurações (por exemplo, habilitar uma nova feature), a alteração é versionada e passa por revisões de *pull request* no GitHub.
3. **Sincronização automática**: o Flipt, faz *polling* periódico das referências no registry. Assim, assim que a mudança é integrada na *branch* principal, a nova configuração é propagada para a instância do Flipt.

4. **Deploy seguro e observável**: como todo o estado de flags está sob controle de versão, é simples reverter configurações (via rollback) e entender exatamente quando e por quem as mudanças foram feitas.